DEVs PROJECT

METODOLOGIA DE LA PROGRAMACIÓN

Contenidos de hoy

- ¿Quees un framework?
- Manejo de boostrap
- Introducción a Django
- Práctica Django I
- Proyecto Integrador final del curso

Frameworks

¿Que es un framework?
Frameworks: Marcos de trabajo para programadores

Un framework, o un marco de trabajo, es una plantilla para desarrollar softwares de manera más rápida y eficiente. Solo necesitas escribir el código que se encargue de la lógica de tu app. No tendrás que reinventar la rueda y gastar tiempo en tareas básicas de desarrollo, como crear clases, manejar objetos y definir funciones típicas. Un framework define la estructura de tu futuro proyecto y proporciona las herramientas necesarias que puedes usar como bloques de construcción.

- **Estructura predefinida**
  Reutilización de código
- **Convenciones y estándares**
  Inversión de control
- **Componentes y herramientas integradas**
- **Facilitación del desarrollo**

Ejemplo:
El framework Bootstrap es utilizado para el desarrollo de interfaces web, proporcionando estilos, componentes y un sistema de diseño adaptable para distintos dispositivos.

Ventajas y desventajas de los frameworks

beneficios de usar frameworks:

Los frameworks proporcionan una estructura clara con una funcionalidad predefinida, lo que permite reducir los costos de desarrollo y lanzar apps rápidamente.

Los frameworks proporcionan una estructura clara con una funcionalidad predefinida, lo que permite reducir los costos de desarrollo y lanzar apps rápidamente.

Gracias a componentes y bibliotecas reutilizables, los desarrolladores pueden usar la base de código y hacer modificaciones sencillas para adaptarla a su proyecto.

Muchos frameworks implementan actualizaciones de seguridad y mejoras continuas.

Los frameworks también tienen sus desventajas:

Estudiar la estructura, convenciones y características de cada framework toma mucho tiempo. Más aún si eres principiante.

La arquitectura predefinida impone ciertas restricciones. Es difícil personalizar una aplicación si el proceso de desarrollo no es flexible.

Algunos frameworks incluyen una gran cantidad de funcionalidades y componentes que pueden resultar innecesarios si tienes una app simple. La sobrecarga de estos elementos lleva a un mayor consumo de recursos.

Tipos y ejemplos de frameworks

Apps móviles

- React Native (JavaScript), permite desarrollar aplicaciones móviles nativas para iOS y Android con una base de código común.
- Flutter (Dart), es un framework de código abierto elaborado por Google. Ayuda a crear interfaces de usuario en aplicaciones para iOS y Android.

Aplicaciones de escritorio

- Electron (JavaScript), usa tecnologías web como HTML, CSS y JavaScript para construir aplicaciones nativas para Windows, macOS y Linux.
- Qt (C++), permite crear aplicaciones multiplataforma para escritorio y dispositivos móviles. Además, ofrece herramientas y bibtiotecas para diseñar interfaces gráficas de usuario.

Videojuegos

- Unity (C#), permite desarrollar juegos 2D y 3D para una variedad de plataformas, incluyendo PCs, consolas y dispositivos móviles.
- Unreal Engine (C++), es un framework avanzado para elaborar juegos realistas de alta calidad.

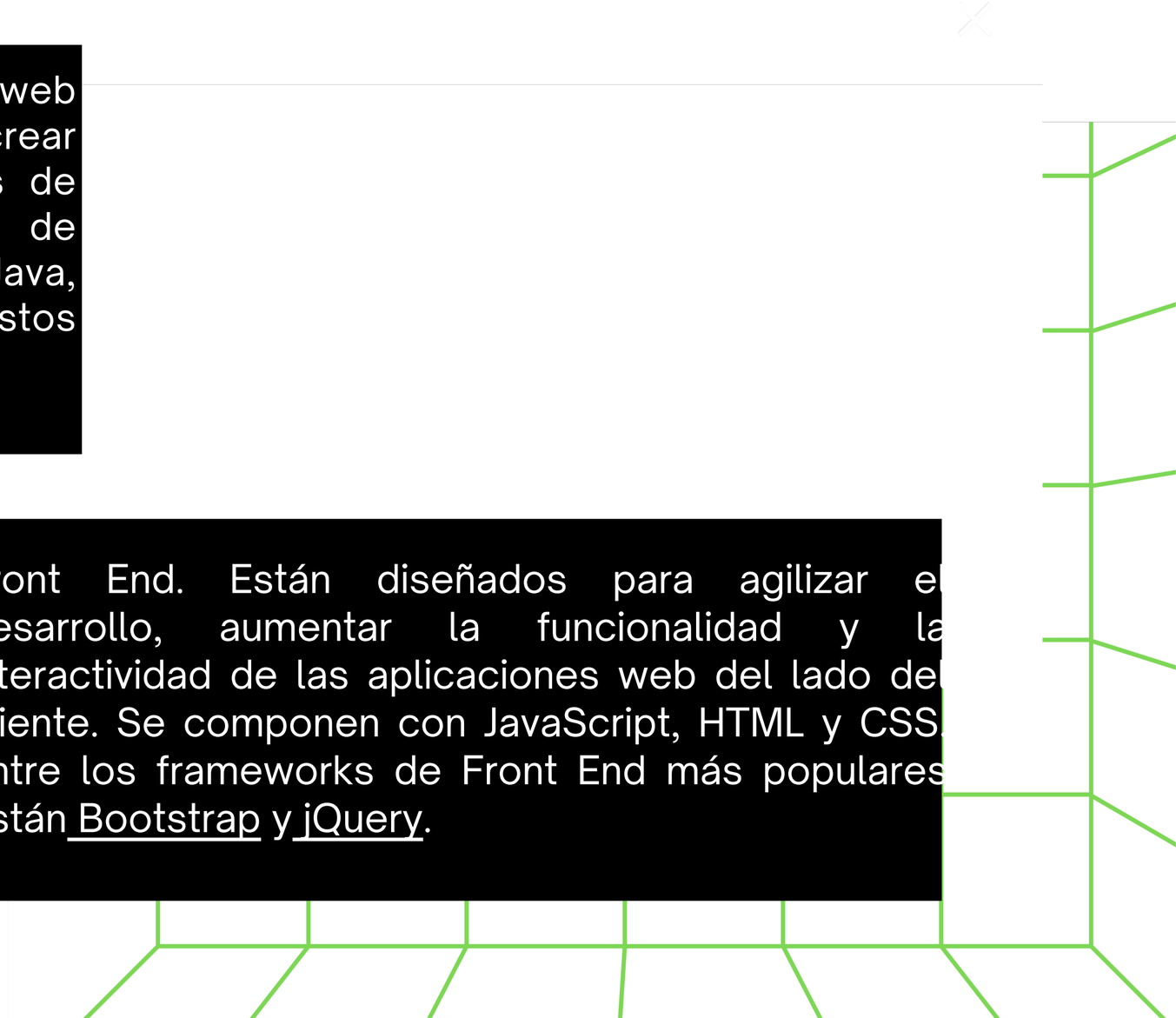
Aplicaciones web

- Django (Python), es un framework de alto nivel que sigue el patrón de diseño Modelo-Vista-Controlador (MVC).
- Angular (JavaScript), está desarrollado por Google para crear aplicaciones web de una sola página (SPA).
- Vue.js (JavaScript), se orienta a la construcción de interfaces interactivas y reutilizables.

Los frameworks de desarrollo web también se puede dividir en dos grupos:

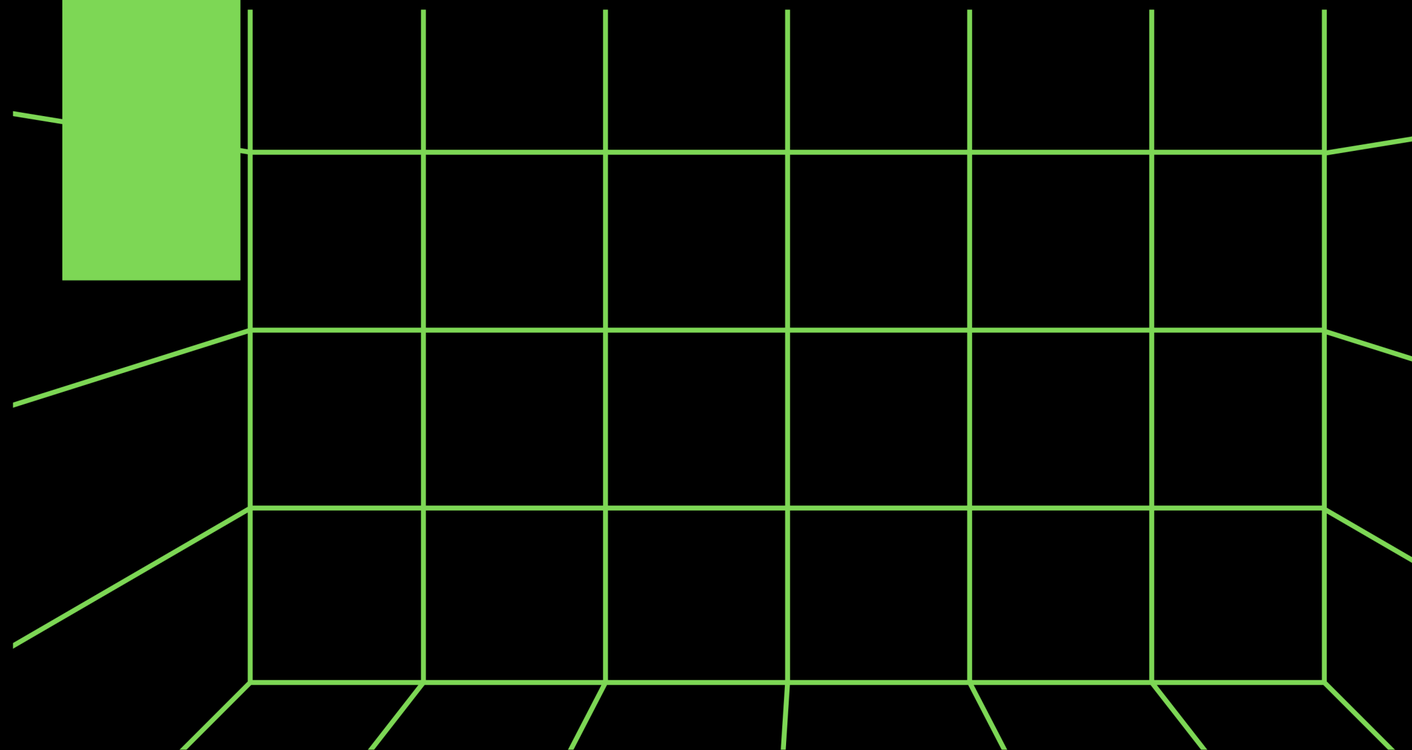
- Back End. Se aplican para desarrollar aplicaciones web del lado del servidor con facilidad. Permiten crear formularios y páginas, así como controlar bases de datos y peticiones HTTP. Usan lenguajes de programación como Python, Ruby, PHP, Javascript, Java, etc. Node.js y Flask son unos ejemplos de estos frameworks.

- Front End. Están diseñados para agilizar el desarrollo, aumentar la funcionalidad y la interactividad de las aplicaciones web del lado del cliente. Se componen con JavaScript, HTML y CSS. Entre los frameworks de Front End más populares están Bootstrap y jQuery.



Bootstrap





Bootstrap

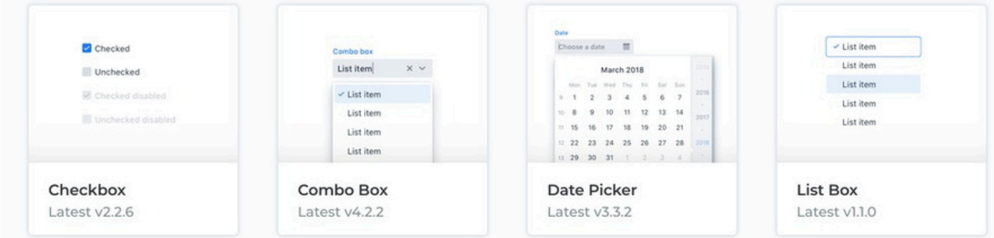
- Es el framework HTML, CSS y JavaScript más popular para crear sitios web y
- móviles responsivos.
  Se puede descargar y usar
- completamente gratis!
  Desarrollado y mantenido por Twitter(Actualmente x) quién la liberó
- como un producto Open Source.
  Tiene una filosofía muy intuitiva para el maquetado de sitios web que puede ser rápidamente aprendida por
- desarrolladores que no vienen del mundo del diseño web.



https://getbootstrap.com/docs

Componentes de bootstrap



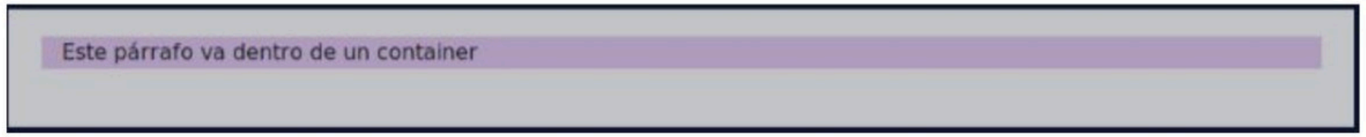


- Accordion
- Alerts
- Breadcrumb
- Button
- Cards
- NavBar
- etc.

Contenedores

- Se utilizan para agrupar elementos, normalmente se usan con el tag div - Cuando se aplica la clase container a un elemento HTML, lo que ocurre es que a ese elemento se le aplica un ancho y un padding determinado y además se coloca en el centro de la página web.

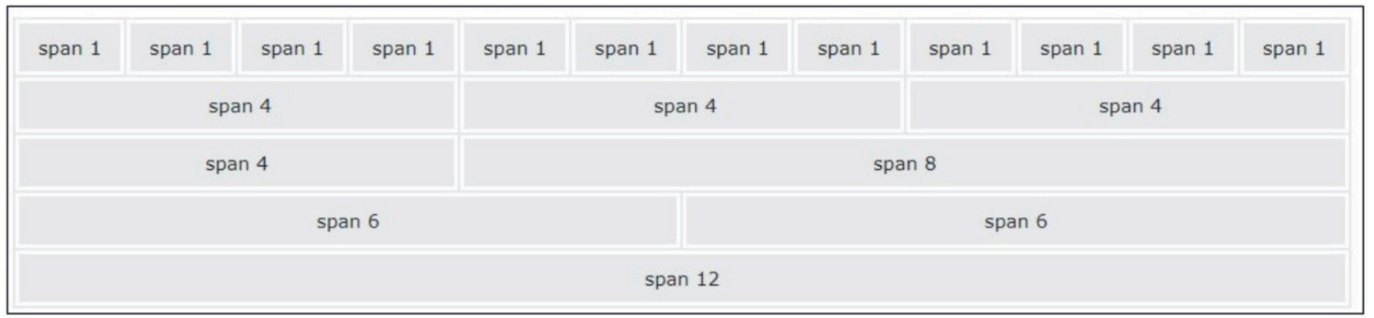
```
<div class="container">
  <p>
    Este párrafo va dentro de un container
  </p>
</div>
```



Sistema de Grillas en Bootstrap

Bootstrap maneja el modo adaptativo mediante un sistema de grillas.

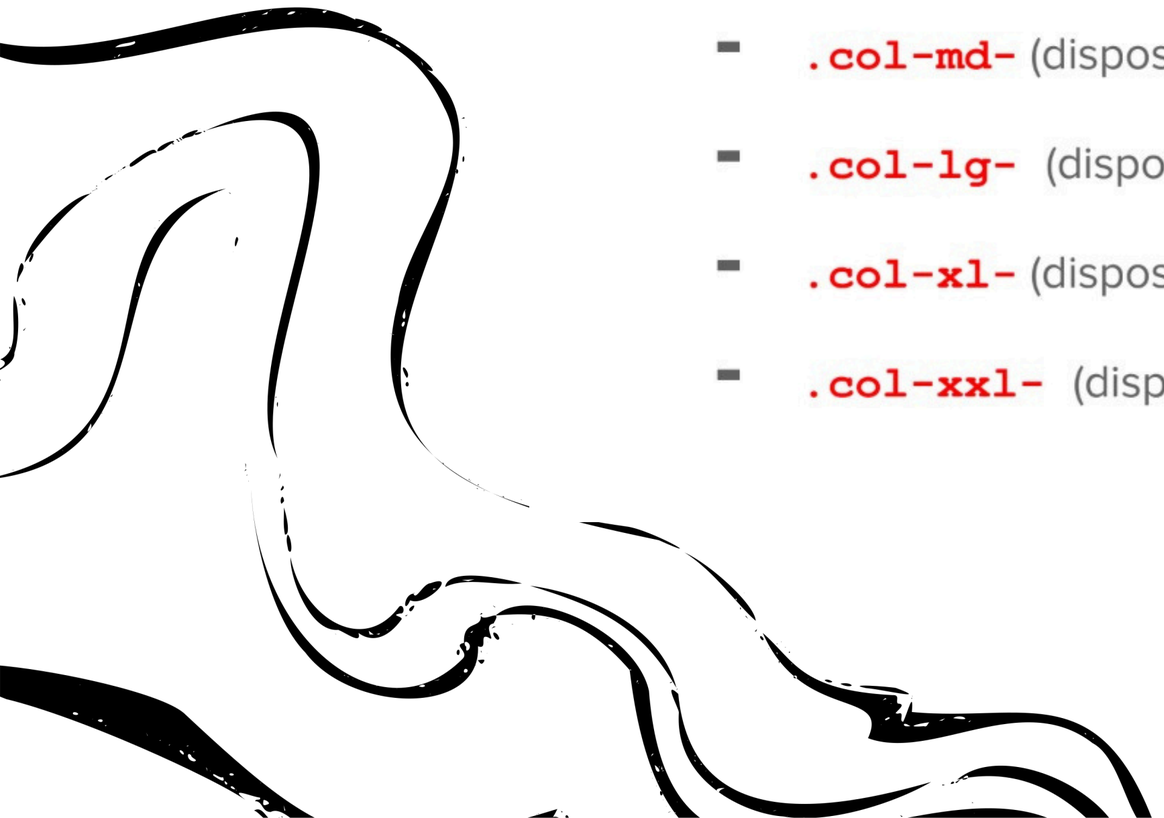
- El sistema de grillas de Bootstrap está construido con flexbox y permite hasta 12 columnas en la página. - Si no desea utilizar las 12 columnas individualmente, puede agrupar las columnas para crear columnas más anchas. - Luego Bootstrap se encarga de colapsar las columnas cuando se accede al sitio desde un dispositivo con una capacidad limitada en cuanto al ancho en píxeles

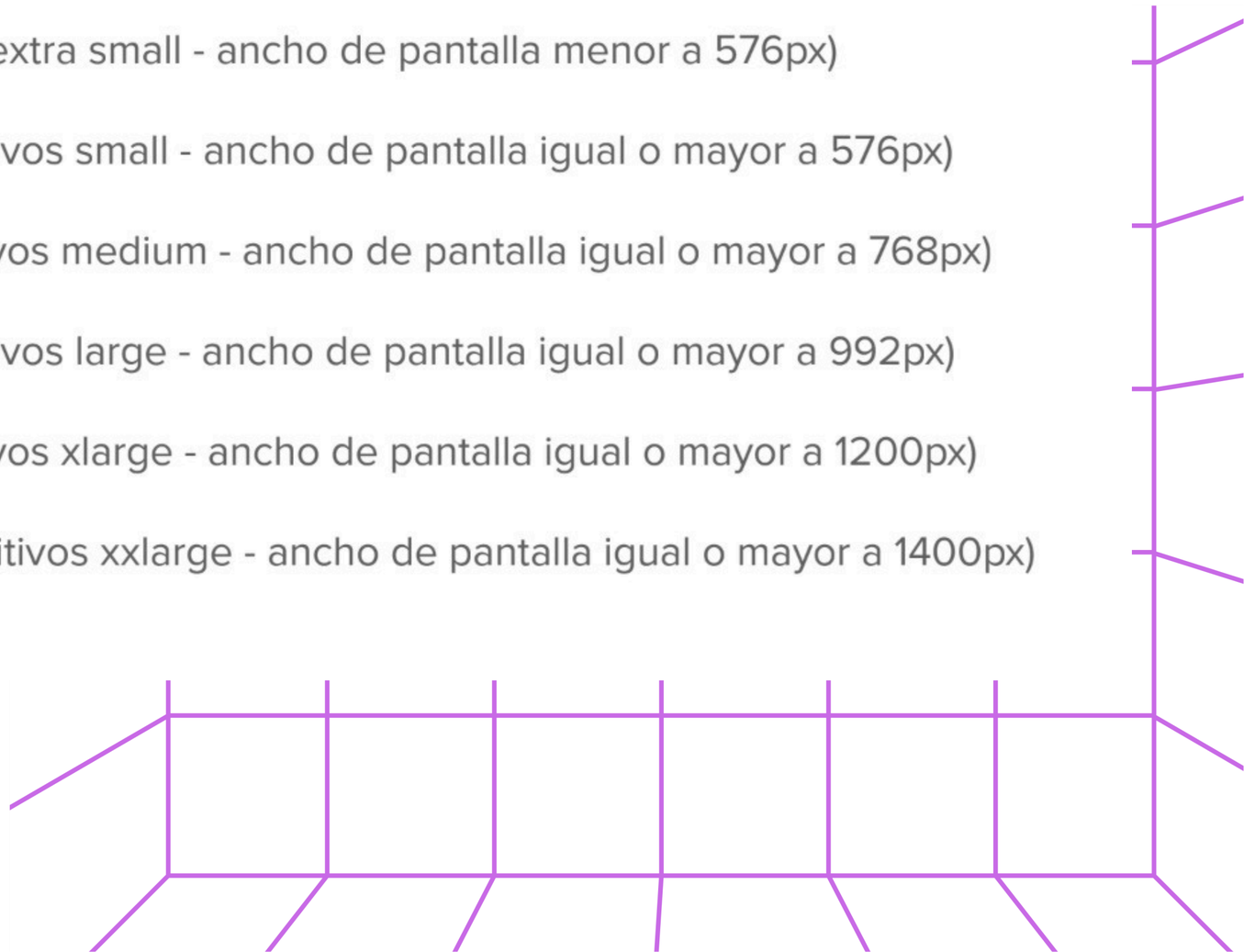




Grid classes

- El sistema de grillas de Bootstrap tiene seis clases css definidas
  - `.col-` (dispositivos extra small - ancho de pantalla menor a 576px)
  - `.col-sm-` (dispositivos small - ancho de pantalla igual o mayor a 576px)
  - `.col-md-` (dispositivos medium - ancho de pantalla igual o mayor a 768px)
  - `.col-lg-` (dispositivos large - ancho de pantalla igual o mayor a 992px)
  - `.col-xl-` (dispositivos xlarge - ancho de pantalla igual o mayor a 1200px)
  - `.col-xxl-` (dispositivos xxlarge - ancho de pantalla igual o mayor a 1400px)





Estructura básica de grid

- Se debe combinar las clases css para los diferentes tamaños de dispositivos

- Se debe combinar las clases css para los diferentes tamaños de dispositivos

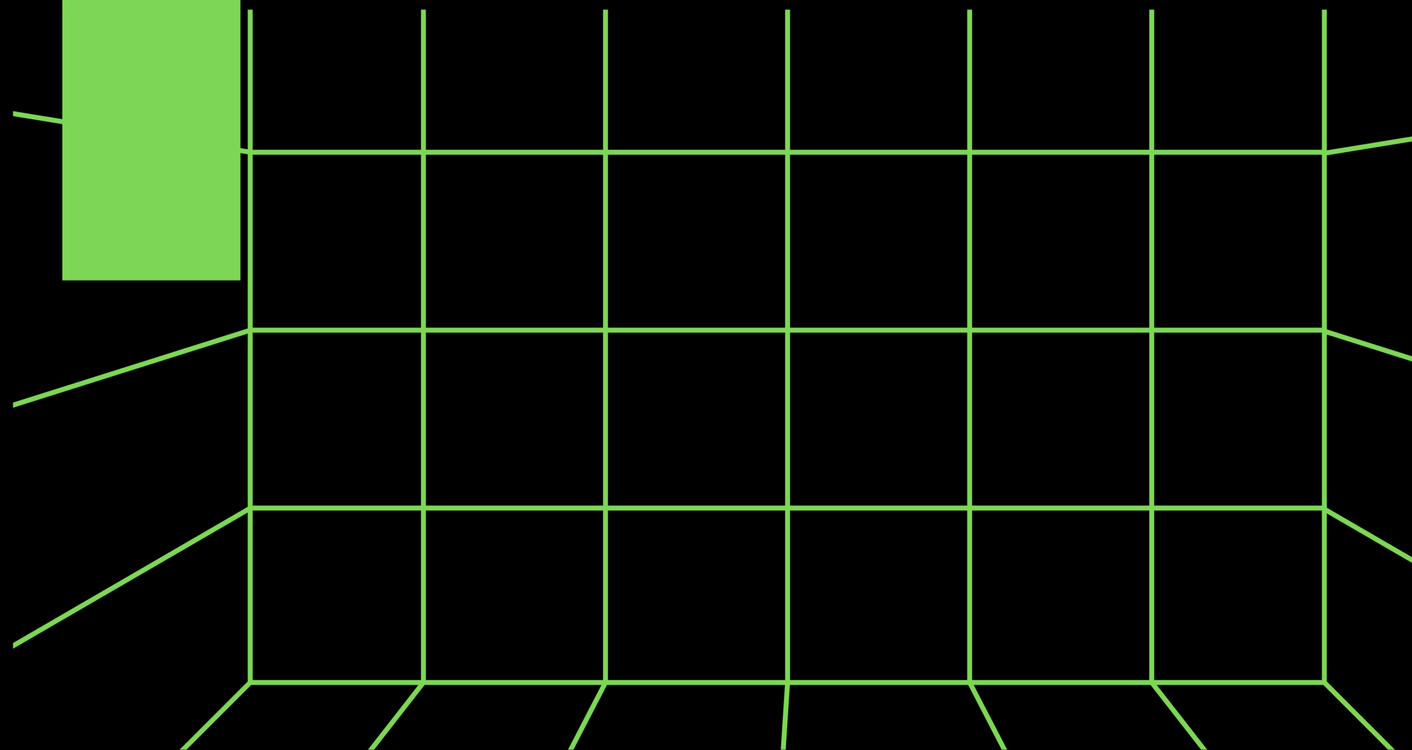
```
<!-- Two columns: 50% width on all screens, except for extra small (100% width) -->
<div class="row">
  <div class="col-sm">1 of 2</div>
  <div class="col-sm">2 of 2</div>
</div>

<!-- Four columns: 25% width on all screens, except for extra small (100% width)-->
<div class="row">
  <div class="col-sm">1 of 4</div>
  <div class="col-sm">2 of 4</div>
  <div class="col-sm">3 of 4</div>
  <div class="col-sm">4 of 4</div>
</div>
```



DJANGO I





¿Qué es Django?

- Es un framework para desarrollo web programado en python, gratuito y libre

- Creado en 2003 cuando los desarrolladores del diario Lawrence Journal-World migraron la arquitectura de su versión online a Python
- Su nombre está dado al guitarrista de jazz Django Reinhardt

- Es ampliamente utilizado por grandes sitios web, tales como Mozilla, Instagram, Bitbucket, etc.
- Promueve la filosofía de desarrollo ágil y extensible aplicando el principio DRY (don’t repeat yourself)



Características más importantes

- Basado en componentes reutilizables (apps)

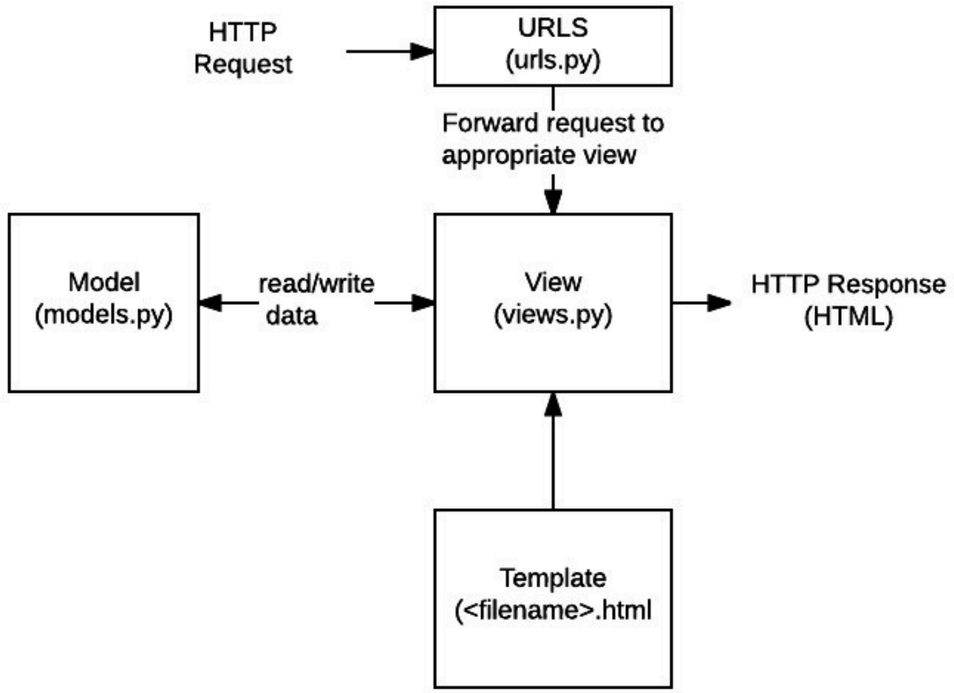
- Un mapeador ORM ((Object-Relational Mapping)) para manejar registros de la BD.
- Un panel de administrador extensible ya creado.
- Un potente sistema de plantillas extensibles con herencia.
- Internacionalización, sesiones, formularios, etc.
- MTV (Modelo-Template-Vista)
Seguridad

Rapidez en el desarrollo

Estructura y organización

Facilidad de integración

Pruebas automatizada



Extensiones recomendadas para VSC

- Code Runner: ejecutar código
- Autopep 8: autoformatear código
- Monokai Pro: personalizar interfaz de usuario



Entornos virtuales

- Es un entorno Python en el que el intérprete Python, las librerías y los scripts instalados en él están aislados de los instalados en otros entornos virtuales, y (por defecto) cualquier biblioteca instalada en un «sistema» Python, es decir, uno que esté instalado como parte de tu sistema operativo



Instalación local vs instalación virtual



Tipos de entornos virtuales en python

- **virtualenv:** crea entornos de Python aislados para bibliotecas de Python
- **pyenv:** para aislar versiones de python
- **pyenv-virtualenv:** plugin para pyenv
- **virtualenvwrapper:** es un conjunto de extensiones de virtualenv. Le brinda comandos para cambiar entre diferentes directorios virtualenv.
- **pipenv:** tiene como objetivo combinar Pipfile, pip y virtualenv en un comando en la línea de comandos, está destinado a ser utilizado al desarrollar aplicaciones Python



Pipenv

- Es la herramienta oficial recomendada para el empaquetado por PyPA (Python Packaging Authority) que apunta a traer lo mejor del empaquetado de otros lenguajes (npm, composer, yarn, etc) al mundo de Python.
- Automáticamente crea entornos virtuales para los proyectos, y permite instalar y desinstalar paquetes de tus archivos Pipfile
- **Pipfile:** archivo utilizado para declarar las dependencias del proyecto.



Instalar pipenv

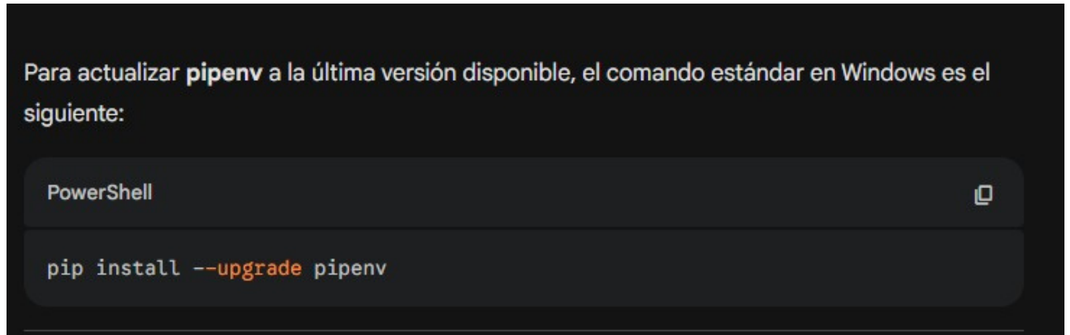
- Previamente hay que actualizar el gestor de paquetes pip:
  python.exe -m pip install --upgrade pip
- Instalar pipenv:

  pip install pipenv

  pipenv --help (validar si se instaló)

- Si no se puede ejecutar pipenv directamente se debe agregar python -m, ejemplo:

  python -m pipenv --help



Crear un entorno virtual con pipenv

1. Crear la carpeta **ejemploPipenv** y luego posicionarse en esa carpeta.
2. **pipenv --python 3.9**: crea el entorno virtual con la versión del entorno del sistema

   pipenv --venv: muestra la ruta del entorno creado.

   pipenv --py: muestra la ruta del interprete de pyton del entorno virtual

1. Comandos disponibles

   pipenv run python: accede al interprete de python del entorno virtual

   pipenv graph: muestra los paquetes del entorno virtual

   pipenv install numpy: instala el paquete numpy en el entorno virtual

   pip list: muestra los paquetes del sistema operativo.

   pipenv uninstall numpy: desinstala el paquete

   pipenv --rm: borra el entorno virtual instalado

Importante: no se puede cambiar el nombre de la carpeta del proyecto, los entornos virtuales quedan enlazados a la carpeta del proyecto donde se creó.



Crear Proyecto djangoUno virtual con pipenv

1. Crear carpeta del proyecto y ubicarse en ella
2. Instalar paquetes necesarios:

   pipenv install django django-ckeditor Pillow pylint pylint-django pylint-celery
3. Verificar entorno creado:

   pipenv --venv
4. Ver lista de comandos:

   pipenv run django-admin
5. Crear proyecto:

   pipenv run django-admin startproject djangoEjemplo
6. Abrir carpeta con VSC



Ejecutar proyecto djangoUno

1. Agregar el siguiente script en archivo **pipfile**

   [scripts]

   djangoUno = "python djangoUno/manage.py runserver"

2. Seleccionar el intérprete de dyango del entorno virtual creado:

   abrir **manage.py** y con botón derecho seleccionar la opción run python file in Terminal

3. Ejecutar script desde la terminal

   python manage.py runserver

4. Ingresar al sitio

   http://127.0.0.1:8000/



Crear la app Core

1. Posicionarse en la consola en el directorio del manage.py:

   python manage.py startapp core

2. Se crea la carpeta core que contiene
   a. views.py: definir las vistas
   b. models.py: definir las clases del modelo
   c. admin.py: registrar los modelos creados
   d. tests.py: diseñar las pruebas unitarias
   e. apps.py: configurar la app



Implementar una vista html

1. Crear la carpeta core/templates/core
2. En la carpeta core crear **home.html**
3. En **Views.py** crear la función home

djangoUno > core > views.py > ...

```python
from django.shortcuts import HttpResponse, render

# Create your views here.

def home(request):
    return render(request, "core/home.html")
```

4. Registrar la app core en settings.py

djangoUno > djangoUno > settings.py > ...

```python
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'core',
```

Implementar una vista html

En core/templates/core/home.html: Agrega un código base (puedes usar el Bootstrap que vimos antes):

core > templates > core > index.html > html

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <link
       href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min
       .css" rel="stylesheet">
6      <title>Inicio - Core</title>
7  </head>
8  <body class="bg-light">
9      <div class="container mt-5">
10         <h1 class="text-primary text-center">Bienvenido a la carpeta
           Core</h1>
11         <p class="lead text-center">Esta vista está cargando desde
           templates/core/index.html</p>
12     </div>
13 </body>
14 </html>
```

Implementar una vista html

En core/templates/core/home.html: Agrega un código base (puedes usar el Bootstrap que vimos antes):

core > templates > core > index.html > html

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <link
       href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min
       .css" rel="stylesheet">
6      <title>Inicio - Core</title>
7  </head>
8  <body class="bg-light">
9      <div class="container mt-5">
10         <h1 class="text-primary text-center">Bienvenido a la carpeta
           Core</h1>
11         <p class="lead text-center">Esta vista está cargando desde
           templates/core/index.html</p>
12     </div>
13 </body>
14 </html>
```

Implementar una vista html

5. Enlazar la vista en **urls.py** del proyecto. Agregamos el import y path de la vista.

- from core import views
- path(‘’, views.home),

Configura el urls.py del proyecto
Busca el archivo urls.py que está en la carpeta principal de tu proyecto (donde está el archivo settings.py) y déjalo así:

djangoEjemplo > urls.py > ...

```
14      1. Import the include() function: from django.urls import include, path
15      2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
16  """
17  from django.contrib import admin
18  from django.urls import path
19  from core.views import home # Importamos tu función de la carpeta core
20
21  urlpatterns = [
22      path('admin/', admin.site.urls),
23      path('', home, name='home') # Esto hace que tu HTML sea la página
        principal
24  ]
25
```

PROYECTO INTEGRADOR

Proyecto Integrador

El Proyecto Integrador tiene como objetivo aplicar de manera práctica los conocimientos adquiridos durante el cursado. En este proyecto, los estudiantes deberán desarrollar una aplicación web utilizando tecnologías vistas en la materia, tales como HTML, CSS, el framework Bootstrap para el diseño de interfaces y el framework Django para el desarrollo del backend. El proyecto permitirá integrar conceptos de estructura de páginas web, estilos visuales, diseño responsivo y desarrollo de aplicaciones web con un framework, fomentando además el trabajo organizado y progresivo durante todo el proceso de desarrollo.

Evaluación y entregas **A lo largo del proyecto se realizarán entregas parciales, en las cuales se evaluará el progreso alcanzado en cada sprint. Estas entregas permitirán verificar el cumplimiento de los objetivos propuestos y brindar retroalimentación para mejorar el desarrollo del proyecto. La entrega final del Proyecto Integrador se realizará el 25 de marzo, fecha en la que finaliza el cursado. En esta instancia se evaluará el proyecto completo, considerando tanto el funcionamiento de la aplicación como la correcta implementación de las tecnologías utilizadas.**



Proyecto Integrador: Metodología de trabajo

El desarrollo del proyecto se organizará mediante sprints, que son periodos de trabajo en los cuales se planifican y desarrollan determinadas funcionalidades del sistema. Cada sprint tendrá objetivos específicos y permitirá avanzar de forma gradual en la construcción del proyecto. Durante el cursado se realizará un sprint intensivo, destinado a avanzar significativamente en el desarrollo de la aplicación, consolidando los conocimientos técnicos y la organización del trabajo.

¿PREGUNTAS?

